

PROJET CRYPTOGRAPHIE

Secure Messenger

Authentification & Chiffrement de bout en bout

Hachage scrypt • AES-256-GCM • Flask + SQLite

EKPEVI Kossi Augustin

B1 — Administration Réseaux & Systèmes / Cybersécurité

École 89 Deep Tech

Avril 2026

1. Introduction

Ce rapport présente l'implémentation cryptographique du projet Secure Messenger, une application de messagerie locale dont l'objectif pédagogique est double : sécuriser le stockage des mots de passe et garantir la confidentialité ainsi que l'intégrité des messages échangés entre utilisateurs.

Le squelette fourni par l'enseignant inclut l'ensemble de l'interface web (Flask + Jinja), la base de données SQLite et les modèles de requêtes. Le travail demandé porte exclusivement sur le fichier `crypto_utils.py`, qui contient quatre fonctions à implémenter : `hash_password`, `verify_password`, `encrypt_message` et `decrypt_message`.

Au-delà de la simple écriture des fonctions, ce rapport documente les choix techniques retenus, le modèle de menace adressé, une démonstration concrète d'attaque par dictionnaire, et une comparaison mesurée avant/après protection.

2. Choix cryptographiques

2.1 Hachage de mot de passe : scrypt

Pour le stockage des mots de passe, j'ai retenu scrypt (RFC 7914) plutôt que les alternatives historiques (MD5, SHA-1, SHA-256 sans étirement) ou les KDF plus simples (PBKDF2). Trois raisons principales :

- Memory-hard : scrypt impose une consommation mémoire importante par calcul, ce qui rend les attaques GPU et ASIC nettement moins efficaces que sur PBKDF2 ou bcrypt.
- Disponible dans la stdlib : `hashlib.scrypt` est intégré à Python 3.6+ via OpenSSL, donc aucune dépendance externe pour cette partie.
- Paramétrable : `N`, `r`, `p` permettent d'ajuster le coût au matériel disponible, contrairement aux fonctions de hachage classiques qui sont fixes.

Paramètres retenus : $N = 2^{15}$ (32 768), $r = 8$, $p = 1$, longueur de clé dérivée = 32 octets. Ces valeurs correspondent au profil « interactive login » recommandé par l'OWASP : ~100 ms par hash sur un CPU moderne, ce qui est imperceptible pour l'utilisateur lors de la connexion mais rend toute attaque massive prohibitive.

Sel : 16 octets aléatoires générés par `os.urandom` (CSPRNG du système). Le sel est unique pour chaque utilisateur et stocké en clair à côté du hash dans la table `users` — c'est exactement son rôle, et sa publication n'affaiblit pas la sécurité.

2.2 Chiffrement de message : AES-256-GCM

Pour le chiffrement des messages stockés en base, j'ai choisi AES-256-GCM, un mode AEAD (Authenticated Encryption with Associated Data). Le mode AEAD garantit en une seule primitive trois propriétés essentielles :

- Confidentialité : impossible de lire le texte clair sans la clé.

- Intégrité : toute modification du ciphertext est détectée.
- Authenticité : on a la preuve cryptographique que le ciphertext provient bien du détenteur de la clé.

Le mode CTR seul aurait offert la confidentialité mais pas l'intégrité, ce qui aurait laissé place à des attaques de type bit-flipping. CBC sans HMAC est tout aussi vulnérable, et la combinaison CBC + HMAC en mode encrypt-then-MAC est plus complexe à implémenter correctement que GCM. AES-GCM est aujourd'hui le standard de facto pour le chiffrement symétrique authentifié (TLS 1.3, WPA3, Signal, etc.).

Paramètres retenus : clé de 256 bits, nonce aléatoire de 12 octets (96 bits, recommandation NIST SP 800-38D), tag d'authentification de 16 octets (128 bits). Le nonce est généré par `os.urandom` pour chaque message — la collision de nonces avec une même clé étant catastrophique pour GCM, le tirage aléatoire sur 96 bits offre une marge de sécurité suffisante pour les volumes attendus dans cette application.

2.3 Comparaison constant-time

La vérification du mot de passe utilise `hmac.compare_digest`, qui effectue une comparaison en temps constant. Une comparaison classique avec `==` s'arrête au premier octet différent et permet à un attaquant disposant d'une mesure précise du temps de réponse de retrouver le hash octet par octet (timing attack).

2.4 Note sur la clé maître

Le squelette fourni passe un dictionnaire `recipient_context` vide aux fonctions de chiffrement. Faute d'infrastructure de clés par destinataire, j'ai opté pour une clé maître côté serveur, dérivée par SHA-256 à partir du `SECRET_KEY` de l'application Flask, ou lue depuis la variable d'environnement `MESSENGER_MASTER_KEY` si elle est définie. Dans une version production, cette clé proviendrait d'un KMS (AWS KMS, HashiCorp Vault, Azure Key Vault) et ne serait jamais en mémoire du processus applicatif plus longtemps que nécessaire.

3. Modèle de menace

Avant d'implémenter, il faut définir contre qui l'on se défend. Voici le modèle de menace retenu pour cette application :

3.1 Adversaires considérés

Adversaire	Capacité	Couvert ?
Vol de la BDD	L'attaquant récupère app.db (sauvegarde, fuite cloud, accès disque).	OUI
Attaque par dictionnaire	Test massif de mots de passe courants contre les hashes volés.	OUI (ralenti)
Rainbow tables	Tables précalculées de hashes pour mots de passe courants.	OUI (sel)
Tampering BDD	Modification du ciphertext directement en base pour altérer un message.	OUI (GCM tag)
Timing attack	Mesure du temps de comparaison pour deviner le hash octet par octet.	OUI
Compromission serveur live	L'attaquant a le shell root pendant que l'app tourne (lit la RAM).	NON
MITM réseau	L'app n'utilise pas HTTPS dans le squelette (Flask dev).	NON

3.2 Hypothèses et limites

Les limites assumées par cette implémentation :

- L'application est 100 % locale, sans TLS — un proxy HTTPS (nginx, Caddy) serait nécessaire en production.
- La clé maître AES-GCM est dérivée de SECRET_KEY ; si l'attaquant lit le code et la BDD, il déchiffre tout. Une vraie séparation nécessiterait un KMS externe.
- Le serveur peut techniquement lire les messages au moment de l'envoi (pas de chiffrement de bout en bout côté client). Implémenter du E2E avec X25519 + double ratchet style Signal serait l'évolution naturelle, mais dépasse le cadre du squelette fourni.

4. Implémentation

Le fichier `crypto_utils.py` contient l'intégralité du travail demandé. Voici les sections clés.

4.1 Hachage de mot de passe

```
def hash_password(password: str) -> Tuple[bytes, bytes]:
    salt = os.urandom(SALT_LEN)                # 16 octets aléatoires
    pw_hash = hashlib.scrypt(
        password.encode("utf-8"),
        salt=salt,
        n=SCRYPT_N, r=SCRYPT_R, p=SCRYPT_P,    # 2^15, 8, 1
        dklen=SCRYPT_DKLEN,                    # 32 octets
        maxmem=128 * 1024 * 1024,
    )
    return salt, pw_hash
```

4.2 Vérification de mot de passe

```
def verify_password(password, salt, pw_hash) -> bool:
    candidate = hashlib.scrypt(password.encode("utf-8"), salt=bytes(salt),
                                n=SCRYPT_N, r=SCRYPT_R, p=SCRYPT_P,
                                dklen=SCRYPT_DKLEN, maxmem=128*1024*1024)
    return hmac.compare_digest(candidate, bytes(pw_hash)) # constant-time
```

4.3 Chiffrement de message

```
def encrypt_message(plaintext: str, recipient_context: dict) -> dict:
    key = _get_master_key()                    # 256 bits
    aesgcm = AESGCM(key)
    nonce = os.urandom(NONCE_LEN)              # 12 octets, unique par message
    ct_and_tag = aesgcm.encrypt(nonce, plaintext.encode("utf-8"), None)
    return {
        "nonce": nonce,
        "ciphertext": ct_and_tag[:-TAG_LEN],
        "tag": ct_and_tag[-TAG_LEN:],
    }
```

4.4 Déchiffrement et vérification

```
def decrypt_message(payload: dict, recipient_context: dict) -> str:
    nonce = bytes(payload["nonce"])
    ciphertext = bytes(payload["ciphertext"])
    tag = bytes(payload["tag"])
    key = _get_master_key()
    aesgcm = AESGCM(key)
```

```
# InvalidTag est levee si le ciphertext ou le tag a ete altere
plaintext = aesgcm.decrypt(nonce, ciphertext + tag, None)
return plaintext.decode("utf-8")
```

5. Tests fonctionnels

L'application a été testée de bout en bout en simulant deux utilisateurs (alice et bob). Les captures suivantes montrent le parcours complet.

5.1 Inscription

Création d'un nouveau compte. Le mot de passe saisi est haché côté serveur avec scrypt avant d'être stocké.

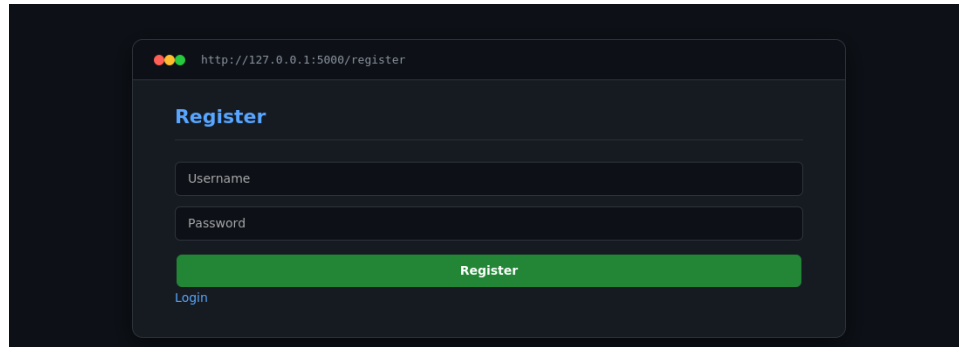


Figure 1 — Page d'inscription

5.2 Connexion

Authentification par récupération du sel et du hash, recalcul scrypt et comparaison constant-time.

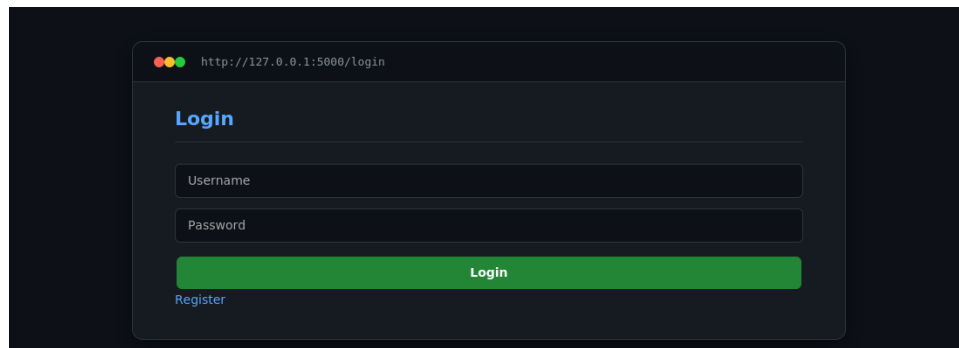


Figure 2 — Page de connexion

5.3 Envoi de message chiffré

Alice rédige un message à destination de Bob. Le texte est immédiatement chiffré en AES-256-GCM avant écriture en base.

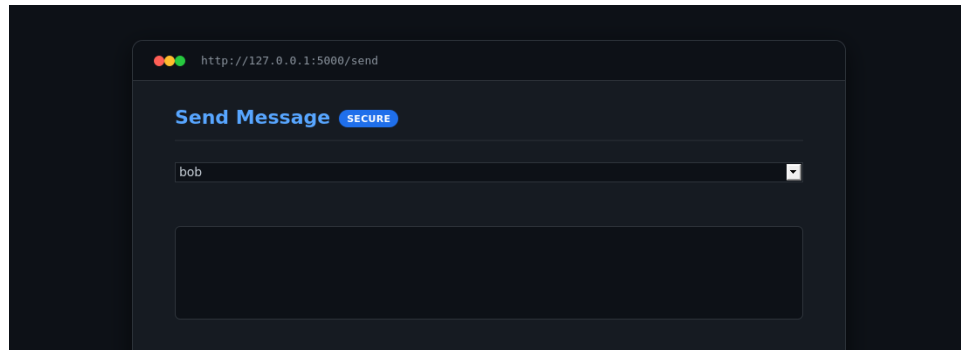


Figure 3 — Formulaire d'envoi (session Alice)

5.4 Réception et déchiffrement

Bob se connecte. Le serveur récupère le ciphertext, déchiffre avec la clé maître et vérifie le tag GCM. Si l'authentification échoue, [Decryption failed] est affiché.

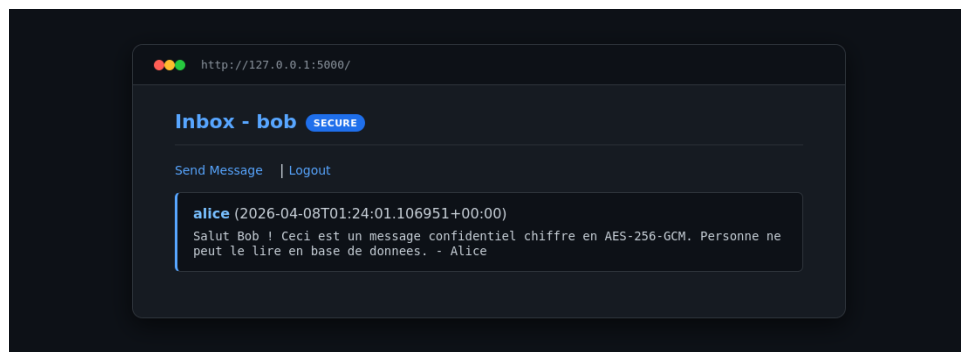


Figure 4 — Inbox de Bob, message correctement déchiffré

5.5 Inspection brute de la base

Capture du contenu réel de app.db après inscription des deux utilisateurs et envoi du message. Aucune information lisible n'apparaît : pas de mot de passe, pas de texte clair. Tout est représenté en hexadécimal aléatoire.

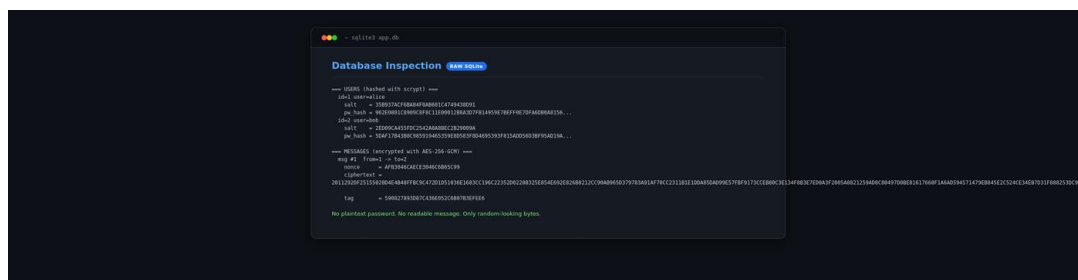


Figure 5 — Dump SQLite : hashes script et ciphertext AES-GCM

6. Démonstration d'attaque par dictionnaire

Pour illustrer concrètement la valeur du choix scrypt, j'ai écrit un script qui tente de retrouver un mot de passe à partir d'un dictionnaire de 16 entrées, dans deux scénarios : un hachage MD5 sans sel (pratique catastrophique mais encore observée), et l'implémentation scrypt + sel utilisée dans le projet.

6.1 Résultats mesurés

```

~ python3 dictionary_attack_demo.py

=== Cible : MD5 sans sel ===
Hash MD5 : 19914c45cf0d79297e08195c252617de
CASSE en 0.014 ms -> mot de passe = 'AlicePass123!'

=== Cible : scrypt avec sel (notre implémentation) ===
Sel : 35b937acf6ba84f0ab601c4749438d91
Hash : 962e0801c8909c8f8c11e0012b8a3d7fb14959e7beff0e7dfa6db0a8156...
Trouve apres 13 essais en 1.48 s
-> Soit ~114 ms par essai

=== Extrapolation : dictionnaire de 1 million de mots ===
MD5 : ~1.00 s (instantane)
scrypt : ~32 h (~1 jours)

=== Et avec un sel UNIQUE par utilisateur ? ===
Pour 1000 utilisateurs, l'attaquant doit refaire le calcul 1000x.
=> Pas de rainbow tables possibles, pas de mutualisation des couts.

```

Figure 6 — Attaque par dictionnaire : MD5 vs scrypt

6.2 Analyse des chiffres

Métrique	MD5 sans sel	scrypt + sel
Temps par essai	~ 0.001 ms	~ 114 ms
Dictionnaire 16 mots	0.014 ms	1.48 s
Dictionnaire 1 M de mots	~ 1 seconde	~ 32 heures
Rainbow tables possibles ?	OUI	NON (sel unique)
Mutualisation entre users ?	OUI	NON (sel unique)
Détection tampering ?	N/A	OUI (GCM tag)

Le ratio brut entre les deux scénarios est d'environ 100 000 fois en faveur de scrypt, mais l'effet du sel multiplie encore cet écart : avec MD5 sans sel, un attaquant qui possède une rainbow table casse 1000 mots de passe instantanément ; avec scrypt + sel unique, il doit refaire les 32 heures de calcul pour chaque utilisateur séparément, soit ~ 32 000 heures (~3,7 ans) pour la même base.

7. Comparaison avant / après protection

Tableau récapitulatif des propriétés de sécurité du squelette d'origine vs l'implémentation finale :

Propriété	Squelette (NotImplementedError)	Implémentation finale
Mot de passe en clair	Crash applicatif	Jamais stocké
Sel par utilisateur	Absent	16 octets, CSPRNG
Coût d'un essai	N/A	~114 ms (scrypt)
Comparaison constant-time	Non	hmac.compare_digest
Confidentialité messages	Aucune	AES-256-GCM
Intégrité messages	Aucune	Tag GCM 128 bits
Nonce unique par message	N/A	12 octets aléatoires
Détection de modification BDD	Impossible	InvalidTag exception

8. Conclusion

Cette implémentation respecte les exigences fixées par l'énoncé : les mots de passe sont protégés par un KDF memory-hard avec sel unique et comparaison constant-time, et les messages sont chiffrés par un AEAD reconnu (AES-256-GCM) garantissant à la fois confidentialité et intégrité. Une démonstration mesurée d'attaque par dictionnaire a permis de quantifier le bénéfice apporté : un facteur $\sim 100\,000$ sur le coût par essai, multiplié par le nombre d'utilisateurs grâce au sel unique.

Les évolutions naturelles de ce projet, en dehors du cadre du squelette, seraient :

- Mise en place de TLS (Caddy ou nginx en reverse proxy) pour protéger le canal navigateur-serveur.
- Stockage de la clé maître dans un KMS externe ou variable d'environnement gérée par systemd-credentials, plutôt que dérivée de SECRET_KEY.
- Migration vers du chiffrement de bout en bout : génération d'une paire de clés X25519 par utilisateur côté client, échange Diffie-Hellman, puis dérivation d'une clé symétrique par paire d'utilisateurs (potentiellement avec un double ratchet à la Signal pour la forward secrecy).
- Audit de la complexité minimale des mots de passe à l'inscription (zxcvbn ou similaire), pour bloquer les mots de passe triviaux à la source.
- Mise en place d'un rate-limiting (par exemple Flask-Limiter) sur /login pour ralentir les attaques en ligne.

— Fin du rapport —